

Aula 5 — Temporizadores e razão cíclica

Quando o processador deixa de ser o relógio

Microcontroladores — DENE/UFMT

OBJETIVOS

- Justificar a existência de um temporizador a partir do custo das esperas bloqueantes dos encontros 3 e 4.
- Calcular o pré-carregamento de um temporizador para um intervalo dado, a partir de T_{cy} e do divisor de entrada.
- Distinguir contagem de tempo por instrução, por indicador consultado e por interrupção, e dizer o que cada uma custa.
- Derivar período e resolução de um sinal modulado por largura de pulso a partir de PR2 e do divisor do Timer2.
- Decidir a frequência de comutação de um atuador a partir da constante de tempo da carga, e reconhecer quando a razão cíclica **não** equivale a uma tensão média.
- Estimar a autonomia de um sistema alimentado a pilha a partir do ciclo útil, e identificar o que limita essa autonomia na prática.
- Distinguir os dois usos do mesmo módulo PWM — entregar energia e sintetizar frequência — e determinar a faixa de frequências que o CCP alcança nesta plataforma.

1 Tudo que este curso fez com tempo até agora

Encontro	Mecanismo	O que o processador estava fazendo
0	Laço vazio contado	Executando instruções que não fazem nada
2	Contagem de ciclos	O mesmo, agora com o número certo
3	<code>__delay_us</code> , espera cega	Nada, por 5 200 ciclos
4	<code>while (ADCON0bits.GO)</code>	Nada, por 60 ciclos

Quatro mecanismos, uma única técnica: **gastar instruções até que o tempo passe**.

Isso funciona e tem um preço que a aula 3 já cobrou. O bloqueio de 1,3 ms da atualização de tela é maior que o meio período de 1,14 ms do lá de 440 Hz — não dá para tocar uma nota e escrever na tela com esta técnica, em nenhuma ordem.

O temporizador é a primeira peça do chip que conta tempo **sem** o processador.

2 Um contador que anda sozinho

Um temporizador é um registrador que incrementa por conta própria, a cada pulso de uma fonte de contagem, sem executar instrução nenhuma. Quando passa do valor máximo, ele volta a zero e **levanta um indicador** — um bit num SFR.

CONCEITO

O processador não precisa acompanhar a contagem. Precisa apenas, de tempos em tempos, olhar o indicador.

O PIC18F4550 tem quatro: Timer0 e Timer1 (16 bits), Timer2 (8 bits, com comparador de período) e Timer3 (16 bits). Neste curso interessam dois — Timer0 para marcar intervalos e Timer2 porque é ele que sustenta o PWM.

2.1 A conta do pré-carregamento

A fonte de contagem interna é $F_{osc}/4$, ou seja, um incremento por ciclo de máquina: 250 ns. Um contador de 16 bits estoura em 65 536 contagens, o que dá 16,4 ms.

Para um intervalo **menor**, não se espera o estouro desde zero: parte-se de um valor mais alto.

Para 10 ms, contando a 250 ns:

CONCEITO

$$N = 10 \text{ ms} / 250 \text{ ns} = 40000 \text{ contagens}$$

$$\text{pré-carga} = 65536 - 40000 = 25536 = 0x63C0$$

Escreve-se 25536 em TMR0, e o estouro acontece exatamente 40000 ciclos depois.

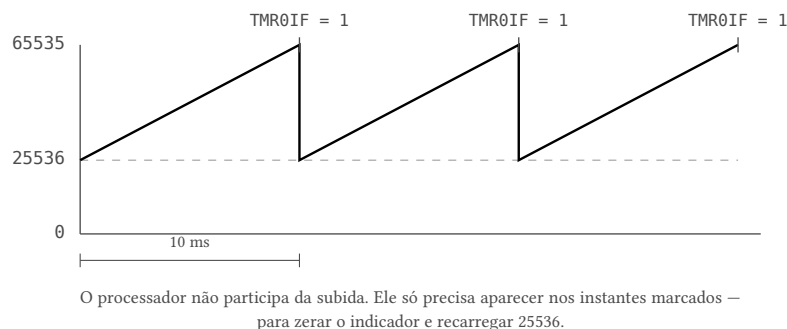


Figura 1: Um intervalo de 10 ms, três vezes. O tempo passa na diagonal; o processador só é convocado nas marcas.

O recarregamento não é gratuito, e o erro que ele introduz é **acumulativo**.

ATENÇÃO

Entre o estouro e a escrita de TMR0 passam-se alguns ciclos — detectar o indicador, entrar no trecho de tratamento, escrever os dois bytes. O contador continuou contando durante esses ciclos, e eles se somam ao intervalo seguinte.

Dez milissegundos viram 10,0025 ms, e depois de uma hora o relógio atrasou nove segundos. É o mesmo tipo de dívida do laço contado do encontro 0 — e a solução é a mesma de sempre: contar os ciclos e descontá-los da pré-carga.

2.2 O código

```
/* 10 ms a 250 ns por contagem: 40 000 contagens. */
#define PRECARGA 25536u /* 65536 - 40000 */

static void t0_recarregar(void)
{
    TMR0H = (uint8_t)(PRECARGA >> 8); /* byte alto vai para um buffer */
    TMR0L = (uint8_t)(PRECARGA & 0xFF); /* a escrita aqui transfere os dois */
}

void t0_iniciar(void)
{
    T0CON = 0x88; /* ligado, 16 bits, fonte interna, sem divisor */
    t0_recarregar();
    INTCONbits.TMR0IF = 0;
}
```

A ordem das duas escritas de t0_recarregar não é estilo. TMR0H não é o byte alto do contador: é um **buffer**. Escrever nele não muda nada; a escrita em TMR0L é que transfere os dezesseis bits de uma vez.

ATENÇÃO

Escrever TMR0L primeiro carrega o byte baixo novo com o byte alto **anterior**, e o contador parte de um valor que ninguém pediu. O erro é intermitente, porque depende de qual era o valor anterior.

A leitura tem a simetria oposta: lê-se TMR0L primeiro, e é essa leitura que congela o byte alto no buffer.

E o laço que consulta:

```

for (;;) {
    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;
        t0_recarregar();
        passou_10ms();
    }

    ler_sensor();
    atualizar_display();          /* pode custar 1,3 ms */
}

```

Para descontar o erro acumulativo da seção anterior, some à pré-carga o número de ciclos gastos entre o estouro e a escrita — `#define PRECARGA (25536u + COMP)`, com COMP medido, não chutado. É a mesma técnica da constante de compensação do gerador de melodia.

NOTA

2.3 Três formas de saber que o tempo passou

Forma	Custo	Quando serve
Laço contado	100% do processador	Atrasos curtos e raros, onde nada mais acontece
Consultar o indicador	Uma leitura por passagem do laço	Quando o laço principal é rápido e previsível
Interrupção	Alguns ciclos por evento	Quando o laço principal pode demorar — encontro 7

A forma do meio é a que o R6 usa, e ela tem uma condição de validade que vale enunciar: **o laço principal precisa dar a volta em menos de um intervalo**. Se uma atualização de tela de 1,3 ms cair no meio de um laço que deveria consultar o indicador a cada 10 ms, ainda cabe. Se o intervalo fosse 1 ms, não caberia — e o indicador seria lido depois de já ter estourado duas vezes.

NOTA

Perder estouros é silencioso. O programa continua rodando e o tempo é que fica errado.

3 Razão cíclica

Ligar e desligar um atuador rápido o bastante entrega a ele uma fração da energia, sem nenhum componente analógico no caminho.

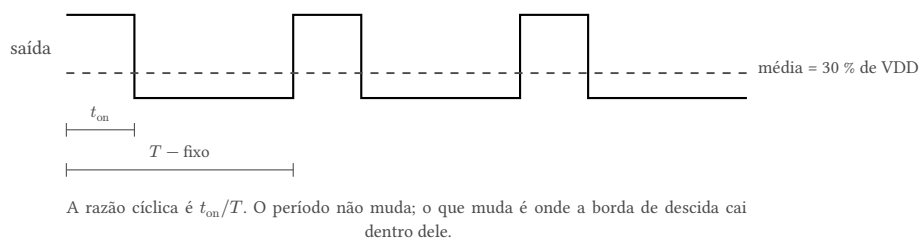


Figura 2: A razão cíclica é a única grandeza que muda. Frequência e amplitude ficam fixas.

A razão cíclica só equivale a uma tensão média se a carga for lenta comparada ao período.

CONCEITO

O que integra o sinal não é o microcontrolador: é a carga. A inércia mecânica da ventoinha, a inércia térmica do aquecedor e a persistência da retina fazem o trabalho. Se a carga responder mais rápido que o período, ela não vê uma média — vê exatamente o que existe, um interruptor batendo.

Carga	Constante de tempo	Frequência mínima razoável
LED	olho: 20 ms	acima de 100 Hz, ou se vê a cintilação
Ventoinha	inércia do rotor: 100 ms	algumas centenas de Hz bastam

Aquecedor	inércia térmica: dezenas de segundos	qualquer coisa acima de 1 Hz
Relé	contato mecânico: ms, e desgasta	nunca

Relé não faz PWM. Cada comutação é um evento mecânico com vida útil contada, e comutar a 1 kHz destrói o contato em minutos.

ATENÇÃO

O aquecedor da bancada, se acionado por relé, é uma saída de duas posições — e a oscilação em torno do setpoint que o R9 vai medir é consequência direta disso. A alternativa é uma chave eletrônica, que é assunto do encontro 8.

3.1 Período e resolução saem do mesmo registrador

O módulo CCP em modo PWM usa o Timer2 como base. O período não é programado em segundos: é programado em contagens, no registrador PR2.

$$T_{\text{PWM}} = (\text{PR2} + 1) \cdot 4 \cdot T_{\text{osc}} \cdot \text{divisor}$$

CONCEITO

A 16 MHz, $4 \cdot T_{\text{osc}} = 250 \text{ ns}$. Com PR2 = 255 e divisor 16:

$$T_{\text{PWM}} = 256 \cdot 250 \text{ ns} \cdot 16 = 1024 \mu\text{s} \rightarrow 976,6 \text{ Hz}$$

E a resolução da razão cíclica, em bits, é

$$\log_2(4 \cdot (\text{PR2} + 1)) = \log_2 1024 = 10 \text{ bits}$$

Com esses valores sai uma coincidência conveniente: o valor de dez bits da razão cíclica, escrito em CCPR1L e nos dois bits de CCP1CON, vale exatamente o tempo ligado **em microssegundos**. Razão cíclica 512 é meio período, e é 512 μs .

NOTA

Vale usar isso em aula e vale desconfiar dela depois: a coincidência morre se alguém mexer no divisor ou em PR2.

Período e resolução são o mesmo botão. Aumentar a frequência exige diminuir PR2, e diminuir PR2 corta bits de razão cíclica.

CONCEITO

Com PR2 = 124 e o mesmo divisor, o período cai para 500 μs — 2 kHz —, e a resolução cai para $\log_2 500 \approx 8,97$ bits. Não existe escolha que dê as duas coisas.

```
void pwm_iniciar(void)
{
    PR2      = 255;           /* 256 contagens por periodo */
    T2CON     = 0x06;         /* Timer2 ligado, divisor 1:16 */
    CCP1CON   = 0x0C;         /* modo PWM */

    LATCbits.LATC2 = 0;       /* LAT antes de TRIS: encontro 2 */
    TRISCbits.TRISC2 = 0;     /* RC2 = CCP1 = COOLER */
}

/* razao: 0 a 1023. Com estes ajustes, o valor tambem e o tempo
   ligado em microssegundos. */
void pwm_razao(uint16_t razao)
{
    if (razao > 1023u) {
        razao = 1023u;
    }
    CCPR1L = (uint8_t)(razao >> 2);           /* 8 bits altos */
    CCP1CON = (uint8_t)((CCP1CON & 0xCF)
        | (uint8_t)((razao & 0x03u) << 4)); /* 2 bits baixos */
}
```

Os dois bits menos significativos da razão cíclica não moram em CCPR1L: moram em CCP1CON, misturados com os bits que selecionam o modo. Daí a máscara 0xCF, que preserva tudo menos esses dois.

NOTA

É a mesma situação da instrução do encontro 2 e do ADCON0 do encontro 4 — uma palavra repartida em campos de significados diferentes. A terceira vez que o curso encontra isso; a essa altura já deveria parecer normal.

Timer2 é um só, e CCP1 e CCP2 dividem ele. Ventoinha e aquecedor, se ambos forem acionados por PWM, terão obrigatoriamente a mesma frequência de comutação.

DIVERGÊNCIA

Isso não é limitação didática: é o tipo de restrição que aparece em projeto real e que obriga a escolher a frequência olhando para as duas cargas ao mesmo tempo.

3.2 Um periférico, dois usos

O mesmo módulo aciona a ventoinha e faz o buzzer apitar, e os dois casos são opostos em tudo que importa.

	Ventoinha	Buzzer
O que carrega a informação	A razão cíclica	A frequência
O que é quase irrelevante	A frequência, desde que a carga integre	A razão cíclica: 50% é o mais alto, e só
O que estraga o resultado	Frequência baixa demais: o rotor pulsa	Frequência errada: a nota sai desafinada
O que a carga faz com o sinal	Integra — vê a média	Não integra — vê cada borda

Na ventoinha, o PWM **entrega energia**: quem interpreta o sinal é a inércia do rotor, e a frequência só precisa ser alta o bastante para que ela consiga integrar.

CONCEITO

No buzzer, o PWM **é o sinal**: a membrana segue cada borda, e o que se ouve é a frequência de comutação. Mudar a razão cíclica muda o timbre e um pouco o volume; mudar a frequência muda a nota.

Duas tarefas sem nada em comum, resolvidas pelo mesmo registrador. É a razão pela qual PR2 e CCP1L parecem arbitrários quando se aprende a fórmula sem perguntar qual dos dois problemas ela resolve.

```
/* Ventoinha: a frequencia e fixa, a razao ciclica e a variavel. */
pwm_iniciar();                /* PR2 = 255, divisor 16: 976,6 Hz */
pwm_razao(307);               /* 30 % de 1023 */

/* Buzzer: a razao ciclica e fixa em 50 %, a frequencia e a variavel. */
static void buzzer_nota(uint16_t hz)
{
    /* T = (PR2+1) * 250 ns * 16 => PR2 = 16e6/(4*16*hz) - 1 */
    uint16_t pr = (uint16_t)(250000UL / hz) - 1u;

    PR2      = (uint8_t) pr;
    CCP1L    = (uint8_t)((pr + 1u) >> 1); /* metade do periodo: 50 % */
    CCP1CON  &= 0xCF;                    /* dois bits baixos em zero */
}
```

E aqui a conta não fecha. Com PR2 no máximo e o maior divisor disponível, o período mais longo que o Timer2 alcança é

ATENÇÃO

$$256 \cdot 250 \text{ ns} \cdot 16 = 1024 \mu\text{s} \rightarrow 976,6 \text{ Hz}$$

Essa é a **menor** frequência que o CCP consegue gerar a 16 MHz. O lá de 440 Hz está abaixo dela, e não há ajuste de registrador que resolva: o divisor do Timer2 só vai até 16, e o pós-divisor não afeta o período do PWM.

Na prática, o CCP deste chip só toca da nota si da quinta oitava para cima.

É por isso que o gerador de melodia do curso não usa o CCP: ele inverte um pino por software, com um atraso contado. E é **por isso** que ele bloqueia o processador, e que a atualização de tela de 1,3 ms atropela o meio período de 1,14 ms do lá.

NOTA

A cadeia inteira agora está fechada: o periférico não alcança a frequência, o software alcança mas bloqueia, e o bloqueio colide com o display. A saída — um temporizador que interrompe e inverte o pino sem que o laço principal saiba — é o encontro 7.

Há um obstáculo a mais, e é de contrato de pinos: CCP1 sai em RC2 e CCP2 em RC1, que são a ventoinha e o aquecedor. Não sobra saída de CCP para o buzzer.

NESTE KIT

Verificar antes do R6: em qual pino está o buzzer do XM118. O código de melodia existente tem um comentário citando RC2 e um `#define` apontando `LATDbits.LATD0` — os dois não podem estar certos.

3.3 A ventoinha é audível

Circula a afirmação de que a ventoinha “não responde à frequência de comutação” porque a inércia do rotor filtra tudo. A parte mecânica sim; o resto não.

DIVERGÊNCIA

O enrolamento do motor sofre magnetostrição na frequência de chaveamento, e o conjunto irradia som nessa frequência. Uma ventoinha em PWM a 1 kHz apita a 1 kHz, com rotação perfeitamente estável.

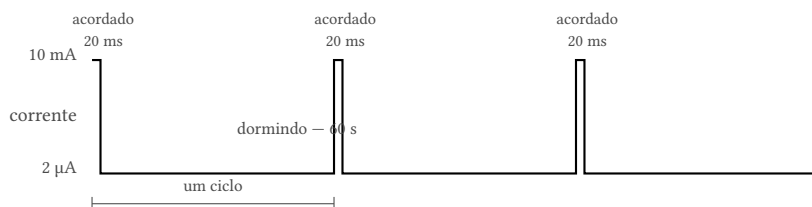
Consequência de projeto: a frequência é escolhida também por critério acústico, e não só elétrico. Comutar acima de 20 kHz resolve o ruído audível e aumenta a perda de comutação na chave.

Verificar na bancada antes de fechar o R6: varrer a frequência de PWM do cooler com razão cíclica fixa e registrar em quais faixas o apito aparece.

4 O caso extremo: o processador desligado

Se o temporizador conta sem o processador, uma pergunta se impõe: por que manter o processador ligado enquanto ele conta?

Uma estação meteorológica mede a cada minuto e passa os outros 59,98 segundos sem ter nada a fazer. A instrução `SLEEP` desliga o oscilador principal e para o núcleo. O consumo cai de miliampères para microampères, e a peça que continua correndo — e que acorda o processador — é um temporizador com oscilador próprio.



A escala vertical não é linear: são quatro ordens de grandeza entre os dois níveis. A média fica muito mais perto do vale do que do pico.

Figura 3: O perfil de consumo de um sensor a pilha. O trabalho acontece nos picos; a conta de autonomia é dominada pelos vales.

A conta da autonomia. Com 10 mA acordado, 2 µA dormindo, 20 ms de trabalho a cada 60 s:

CONCEITO

$$\text{ciclo útil} = 20 \text{ ms} / 60 \text{ s} = 3,3 \cdot 10^{-4}$$

$$I_{\text{média}} = 10 \text{ mA} \cdot 3,3 \cdot 10^{-4} + 2 \text{ µA} \approx 5,3 \text{ µA}$$

Com duas pilhas AA de 2000 mAh, isso dá cerca de 380000 horas — mais de quarenta anos.

O mesmo circuito sem dormir consome 10 mA e dura 200 horas: **oito dias**.

Os quarenta anos são falsos, e a razão é conteúdo.

ATENÇÃO

Nenhuma pilha alcalina dura quarenta anos: a autodescarga a consome em cinco a dez. Quando o ciclo útil fica pequeno o suficiente, **o limite deixa de ser o circuito e passa a ser a química da pilha** — e continuar otimizando firmware não compra mais nada.

Saber onde essa fronteira está é o que separa um projeto de baixo consumo de um projeto que apenas parece cuidadoso.

4.1 O cão de guarda como despertador

O temporizador que acorda o processador não pode depender do oscilador principal, que está desligado. No PIC18F4550 esse papel cabe ao **watchdog** — um contador com oscilador RC próprio, de período nominal de 4 ms, com divisor programável até cerca de 131 s.

NOTA

O watchdog existe originalmente para outra coisa: reiniciar o chip quando o programa trava. Usá-lo como despertador é um segundo emprego, e o modo de reset por watchdog é justamente o que o encontro 8 vai tratar sob a pergunta “por que ele reiniciou?”.

```
void main(void)
{
    configurar();

    for (;;) {
        ligar_sensor();
        int16_t t = ler_temperatura();
        transmitir(t);
        desligar_sensor();

        SLEEP();          /* a execucao continua aqui quando o WDT estourar */
        NOP();
    }
}
```

Duas condições que esta placa não satisfaz, e vale dizê-las em voz alta:

NESTE KIT

O bit de configuração que habilita o watchdog pertence ao bootloader, como PBDEN no encontro 4 — de novo **o código que veio antes do seu**. Se ele estiver desabilitado na gravação, resta o bit SWDTEN de WDTCON, que habilita o watchdog por software, **se** a configuração tiver deixado essa porta aberta. Item a verificar antes do roteiro.

E o XM118 não mede microampères. Ele tem display, relés e conversor USB permanentemente alimentados, e vem da porta USB. O ganho de dormir é real e **invisível** nesta bancada.

Por isso esta seção é argumento, não experimento. O termostato do semestre nunca dorme — ele está ligado na tomada e precisa medir continuamente. Saber por que **ele** não dorme é tão conteúdo quanto saber por que a estação meteorológica dorme.

5 Previsão para o R6

PREVISÃO

P1. Com $PR2 = 255$ e divisor 16, qual frequência você espera medir no pino do cooler? Escreva o número antes de ligar o osciloscópio.

P2. Você vai variar a razão cíclica de 0 a 100% em passos de 10%. Em qual passo espera que o rotor comece a girar? Justifique com o que sabe sobre torque de partida.

P3. Depois de partir, você vai **reduzir** a razão cíclica. O rotor para no mesmo valor em que partiu? Diga sim ou não e por quê.

P4. Meça a frequência com o osciloscópio e compare com P1. Se divergir, o erro está em $PR2$, no divisor, ou na sua hipótese sobre F_{osc} ?

P5. Com razão cíclica em 50%, aproxime o ouvido do cooler. Descreva o que ouve e diga em que frequência.

Leve esta folha preenchida. A previsão é avaliada pelo raciocínio, não por acertar o número.

SEM NOTA

6 Exercícios

Exercício 5.1. Calcule a pré-carga do Timer0 para um intervalo de 2 ms, com fonte interna e sem divisor.

TAREFA

(a) Quantas contagens?

(b) Qual valor escrever em TMR0?

(c) O mesmo intervalo é possível **sem** pré-carga, usando o divisor? Justifique.

Exercício 5.2. Um programa consulta TMR0IF dentro do laço principal, e o laço contém uma atualização de tela de 1,3 ms. O intervalo do temporizador é de 1 ms.

TAREFA

(a) O que acontece?

(b) O sintoma é visível ou silencioso?

(c) Proponha duas correções, e diga qual delas é a do encontro 7.

Exercício 5.3. Você precisa de PWM a 5 kHz no cooler, a 16 MHz.

TAREFA

(a) Escolha PR2 e o divisor do Timer2.

(b) Qual resolução de razão cíclica resulta?

(c) Se o projeto exigir 10 bits de resolução, qual é a maior frequência possível?

Exercício 5.4. Explique por que a razão cíclica de 30% aplicada a um LED, a um aquecedor e a um relé produz três resultados de naturezas diferentes.

TAREFA

Exercício 5.5. O laço principal de um programa faz, em cada volta: ler o ADC (60 ciclos), decidir (poucos ciclos) e atualizar a tela (5200 ciclos). Ele consulta TMR0IF uma vez por volta.

TAREFA

(a) Qual é o menor intervalo de temporizador que esse programa pode medir sem perder estouros?

(b) Se a tela for atualizada apenas quando o valor muda, o número da alínea (a) continua válido?

Exercício 5.6. Um sensor de umidade a pilha acorda, mede e transmite em 40 ms, e dorme o resto do tempo. Acordado consome 12 mA; dormindo, 3 μ A. A pilha tem 1200 mAh.

TAREFA

(a) Calcule a autonomia para um intervalo de 10 s entre medidas.

(b) E para 10 min.

(c) Passar de 10 min para 1 h vale a pena? Justifique com números.

No encontro 6: avaliação integradora I, cobrindo os encontros 0 a 5.

NOTA

E no encontro 7: interrupções. Esta aula deixou duas dívidas do mesmo tipo — o erro acumulativo do recarregamento e a perda silenciosa de estouros. As duas existem porque o processador precisa **aparecer** no instante certo, e até agora a única forma de fazê-lo aparecer era ele mesmo ir olhar.